

Parte Prática

Situações de plágio serão penalizadas com cotação 0, para todas as perguntas, tanto para parte prática como parte teórica, para todos os envolvidos, tanto nesta prova como em todas as provas subsequentes deste ano letivo.

Aquando da submissão da prova, o aluno deve estar ligado na sessão zoom. Os registos de utilização do moodle e do zoom serão utilizados para validar, de forma automática, o cumprimento desta regra. Em caso de incumprimento, a submissão será ignorada.

1 – [65%] Pretende-se criar uma classe **Sequências** que armazena informação sobre sequências de números. Estas sequências de números têm nomes, (por, exemplo, o nome da sequência 1 1 2 3 5 8 13 21 é “Fibonacci”). Os valores das sequências podem ser números de qualquer tipo definido aquando da declaração do objecto em questão (por exemplo, `Sequencias<Integer> s;`). Recorde que `Number` é a classe base de `Integer`, `Double`, etc.

Pretende-se que esta classe suporte o seguinte interface descrito abaixo, **de acordo com a especificação de complexidade indicada**. Deve considerar que **todas** as funcionalidades indicadas são necessárias simultaneamente, portanto é possível que uma funcionalidade tenha implicações relativamente a outras que são especificadas antes ou depois dela. Compete-lhe perceber que implicações serão essas. Se precisar de assumir qualquer coisa relativamente a comportamentos não especificados (por exemplo, situações em que excepções poderiam ser geradas), indique-o claramente na sua resposta.

A especificação dos valores de retorno e tipos de objetos está deliberadamente omissa - deve identificar e utilizar tipos adequados ao pretendido. **Não vale a pena perder o seu tempo a criar soluções que não respeitem as restrições de complexidade indicadas.** Destas, será valorizada apenas a parte relativa à correta identificação dos tipos dos parâmetros e valores de retorno:

- `criaSequencia(nomeSequencia);` Cria uma sequência vazia com o nome indicado. Complexidade **O(log S)** (ou melhor), em que S é o número de sequências.
- `acrescentaValor(nomeSequencia, valor);` **O(1)** – Acrescenta o valor indicado a uma sequência criada anteriormente.
- `getSequencias(nomeSequencia1, nomeSequencia2);` **O(1)** – Este método deve ser construído de forma a garantir que o seguinte código:

```
Sequencia<Integer> si=...
// ...
for (Integer i:si.getSequencias("Fibonacci","Geometrica"))
    { System.out.println(i); }
```

percorre corretamente todos os valores introduzidos nas sequências “Fibonacci” e “Geometrica”, em ordem alternada (ou seja, primeiro o primeiro valor da sequência “Fibonacci”, em seguida o primeiro valor da sequência “Geometrica”, seguindo-se o segundo valor da sequência “Fibonacci”, o segundo da sequência “Geométrica”, etc). Se uma das sequências for mais curta, deve imprimir de forma alternada enquanto possível, passando depois a imprimir apenas o conteúdo em falta da sequência maior.

Se alguma sequência não existir, o comportamento deve ser o mesmo que se obteria se esta fosse uma sequência conhecida mas vazia (ou seja, imprime apenas a outra sequência).

- `removePrimeiro(nomeSequencia)` **O(1)** – remove primeiro valor da sequência indicada (neste caso, o primeiro valor é aquele que foi introduzido à mais tempo e ainda não foi removido).
- `removeSequenciaComMaiorNome()` – Elimina a sequência com maior nome (em caso de empate, pode remover qualquer uma das sequências em questão). Deve realizar esta operação com a melhor complexidade possível, usando de forma adequada as estruturas de dados que conhece.
- `verificaSeSequenciaComeçaCom(nomeDaSequencia, listaValores)` – Recebe uma lista de valores e verifica se a sequência indicada começa com os valores contidos nessa lista. Deve ter complexidade **O(N)**, onde N é o maior valor entre o número de valores da sequência indicada e o número de valores iniciais indicados.
- `removeValoresMaioresQue(String nomeDaSequencia, valor)` – Recebe um valor e remove da sequência indicada todos os que são maiores que o valor recebido. Deve ter complexidade **O(N)**, onde N é o número de valores contidos na sequência.
- `acrescentaSequencias(sequencias)` – Recebe um outro objeto desta classe (Sequencias) que seja compatível e copia todas as sequências nele contidas para este objecto (this). Esta operação deve ter complexidade **O(N log N)** (ou melhor), onde N é igual à soma do número de sequências em ambos os objetos. As sequências copiadas não são clonadas ou duplicadas, sendo “partilhadas” por ambos os objectos – i.e. todos os valores acrescentados a uma sequência de nome “X” num objeto serão visíveis na mesma sequência copiada para o outro objeto.

2 – [35%] Considere uma classe `Arvore` que implementa uma árvore binária de pesquisa.

- Crie um método recursivo *mostraForaDeIntervalo* que recebe dois valores (que definem os limites de um intervalo) e imprime todos os valores da árvore fora do intervalo especificado. O método não deve realizar travessias da árvore desnecessárias. Construa também o respetivo método público não-recursivo.
- Crie um método recursivo *removeRamos* que modifica uma árvore de forma a garantir que não existem nodos com apenas um descendente direito, que sejam eles próprios descendentes direitos e sem irmão esquerdo. Isso é conseguido com a transformação demonstrada na figura. No exemplo da Figura 1, o nodo 20 tem apenas um descendente direito, é um descendente direito e não tem irmãos (do lado esquerdo). Do lado direito, pode observar-se a modificação que deve ser efectuada à árvore para corrigir a situação.

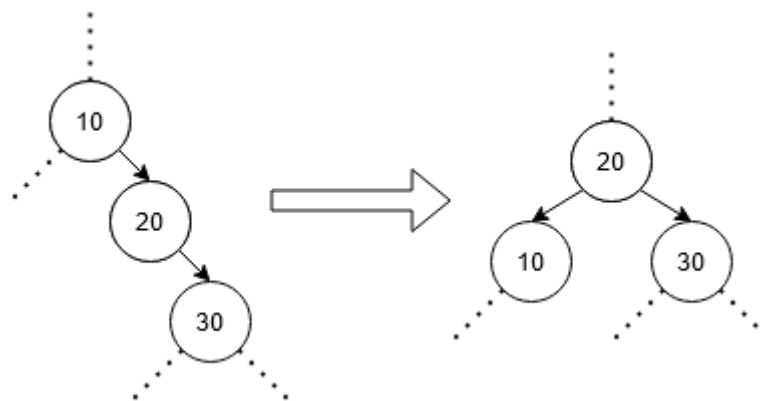


Figura 1 - Transformação pretendida

A Figura 2 apresenta um exemplo de aplicação do método (incluindo vários passos intermédios). A árvore de entrada é representada do lado esquerdo, enquanto que a árvore de saída será a do lado direito.

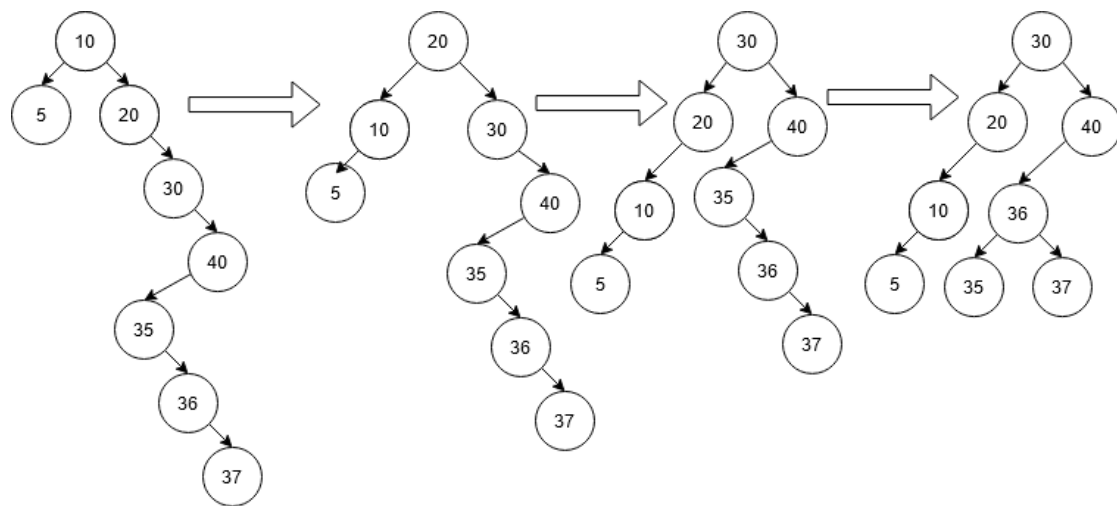


Figura 2 - Exemplo de aplicação